

Lab Sheet 9 – Introduction to CUDA (Google Colab Edition)

SE3082 – Parallel Computing

Semester 2, 2026

Introduction

In this lab you will write and run your first CUDA C++ programs on a real NVIDIA GPU, using **Google Colab**. Colab gives you a free GPU in your browser, so there is nothing to install and nothing to pay for. All the programs from the lecture are here, followed by two exercises where you write the kernel yourself.

By the end of this lab you will be able to:

- switch on a GPU runtime in Colab and confirm it is working with `nvidia-smi`;
- write, compile (`nvcc`) and run a CUDA program from notebook cells;
- explain the host/device pattern: **copy in** → **launch kernel** → **copy out**;
- map blocks and threads onto array elements, in one dimension and in two, with a bounds check.

Exercise	Topic
1	Set up your CUDA environment in Google Colab
2	Your first kernel: adding two numbers
3	Vector addition with 512 blocks and one thread each
4	Vector addition with one block and 512 threads
5	Multiplying two 10,000,000-element vectors (blocks <i>and</i> threads)
6	Multiplying two 10,000 × 10,000 matrices element by element (2D grid)

You need: a Google account and a web browser. Colab menu names change slightly from time to time; if a menu item is not exactly where this sheet says, look for the closest match.

Exercise 1 – Set up your CUDA environment in Google Colab

Step 1 – Create a notebook

1. Open colab.research.google.com and sign in with your Google account.
2. Choose **File** → **New notebook**.
3. Click the notebook name at the top left and rename it to **SE3082_Lab09_<your student ID>**.

Step 2 – Switch on the GPU

1. Choose **Runtime** → **Change runtime type**.
2. Under **Hardware accelerator** select **T4 GPU**, then click **Save**.
3. Colab connects to a machine with a GPU when you run your first cell (the next step).

TIP

A notebook has two kinds of cell: **Text** cells for notes and **Code** cells for code. Use **+ Text** to add a heading such as “Exercise 2” above each exercise so your notebook is easy to read. Run a cell with its play button or **Shift + Enter**. Lines starting with **!** are sent to the Linux shell instead of Python.

Step 3 – Check that the GPU is visible

Colab CODE cell · checks the GPU

```
!nvidia-smi
```

The output should show a table that includes the **GPU name** (normally *Tesla T4*), about 15 GB of memory, and a **CUDA Version** line. If it says the command was not found, or that it failed to communicate with the driver, your runtime has no GPU. Go back to Step 2.

Step 4 – Check the CUDA compiler

Colab CODE cell · checks nvcc

```
!nvcc --version
```

You should see a line such as *Cuda compilation tools, release 12.x*. If the shell cannot find *nvcc*, try *!/usr/local/cuda/bin/nvcc --version* and use that full path in the compile commands below.

Step 5 – Find your GPU's architecture code

The compiler needs to know which GPU generation to build for. Ask the driver for the **compute capability**:

Colab CODE cell · shows GPU name and compute capability

```
!nvidia-smi --query-gpu=name,compute_cap --format=csv
```

Remove the dot to get the architecture code: compute capability **7.5** means *-arch=sm_75*. If the *compute_cap* column is not supported, use this table:

GPU shown by nvidia-smi	Compute capability	Compiler flag
Tesla T4 (free tier, normal case)	7.5	<i>-arch=sm_75</i>
NVIDIA L4	8.9	<i>-arch=sm_89</i>
NVIDIA A100	8.0	<i>-arch=sm_80</i>
NVIDIA H100	9.0	<i>-arch=sm_90</i>

This sheet uses *-arch=sm_75* everywhere. If your GPU is different, change it in every compile cell.

Step 6 – The write → compile → run workflow

Colab has no CUDA editor, so you will keep each program inside a code cell and turn it into a file with the *%%writefile* command. Every exercise follows the same three steps:

	What you do	Cell content
1 Write	Save the program as a <i>.cu</i> file on Colab's disk	<i>%%writefile name.cu</i> on the first line , then the code
2 Compile	Turn the file into an executable	<i>!nvcc -arch=sm_75 name.cu -o name</i>
3 Run	Run the executable	<i>!./name</i>

Steps 2 and 3 can share one cell: *!nvcc -arch=sm_75 name.cu -o name && ./name*. The *&&* means “run the program only if the compile succeeded”.

COMMON MISTAKE

Edit → write → compile → run. If you change the code you must run the *%%writefile* cell *again* and then compile again. Editing the text without doing both means you are still running the old program.

%%writefile must be the very first line of its cell. No blank line or comment above it.

Step 7 – Smoke test: hello from the GPU

Run these two cells to prove the whole toolchain works.

Colab CODE cell · writes hello.cu

```
%%writefile hello.cu
#include <stdio>

__global__ void hello() {
    printf("Hello from GPU thread %d\n", threadIdx.x);
}

int main(void) {
    hello<<<1, 4>>>>();          // 1 block of 4 threads
    cudaDeviceSynchronize();    // wait so the GPU's output is flushed
    return 0;
}
```

Colab CODE cell · compiles and runs

```
!nvcc -arch=sm_75 hello.cu -o hello && ./hello
```

You should see four lines, “Hello from GPU thread 0” to “thread 3”. The lines may appear in **any order**: the four threads run at the same time and nobody controls who prints first.

Step 8 – Colab housekeeping

- **Your code is safe, your files are not.** Colab saves the notebook to your Google Drive, but the **.cu** files and compiled programs live on a temporary disk. If the runtime disconnects (idle time, or a long break), reconnect and use **Runtime → Run all** to rebuild them from your cells.
- **Free GPU time is limited.** When you finish a session choose **Runtime → Disconnect and delete runtime** so the GPU is released. If Colab says no GPU is available, wait and try again later.
- **No cost.** You do not need Colab Pro, a credit card or an AWS account for this lab. Ignore any upgrade prompts.

EXERCISE 1 CHECKLIST

- ☐ The **nvdiia-smi** cell shows a GPU (write down its name and compute capability).
- ☐ The **nvcc --version** cell shows a CUDA release.
- ☐ The **hello** program printed four lines.

Exercise 2 – Simple calculation program from the lecture

This program adds two integers on the GPU using one block containing one thread. Every CUDA program follows the same three steps:

Step	What happens	Code in the program
1	Copy the inputs from host (CPU) memory to device (GPU) memory	<code>cudaMemcpy(..., cudaMemcpyHostToDevice)</code>
2	Launch the kernel on the GPU	<code>add<<<1, 1>>>>(d_a, d_b, d_c)</code>
3	Copy the result from device memory back to the host	<code>cudaMemcpy(..., cudaMemcpyDeviceToHost)</code>

Colab CODE cell · writes ex2_add.cu

```
%%writefile ex2_add.cu
#include <stdio>

// Kernel: this code runs on the device (the NVIDIA GPU)
__global__ void add(int *a, int *b, int *c) {
    *c = *a + *b;
}

// host = CPU, device = GPU
int main(void) {
    int a, b, c;                // host copies of a, b, c
    int *d_a, *d_b, *d_c;      // device copies of a, b, c
    int size = sizeof(int);

    // Allocate space for device copies of a, b, c
    cudaMalloc((void **)&d_a, size);
    cudaMalloc((void **)&d_b, size);
    cudaMalloc((void **)&d_c, size);

    // Set up input values
    a = 2;
    b = 7;

    // 1. Copy the inputs from host to device
    cudaMemcpy(d_a, &a, size, cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, &b, size, cudaMemcpyHostToDevice);

    // 2. Launch the add() kernel on the GPU: 1 block, 1 thread
    add<<<1, 1>>>>(d_a, d_b, d_c);

    // Colab edition: report a failed launch instead of silently printing garbage
    cudaError_t err = cudaGetLastError();
    if (err != cudaSuccess)
        printf("Kernel launch failed: %s\n", cudaGetErrorString(err));

    // 3. Copy the result from device to host
    cudaMemcpy(&c, d_c, size, cudaMemcpyDeviceToHost);
    printf("Result is %d\n", c);

    // Cleanup
    cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
    return 0;
}
```

Colab CODE cell · compiles and runs

```
!nvcc -arch=sm_75 ex2_add.cu -o ex2_add && ./ex2_add
```

Expected output: **Result is 9**

What is new in this Colab edition: the lines after the kernel launch call `cudaGetLastError()`. A kernel launch that fails (for example because of a wrong `-arch` flag) does not stop the program or print anything by itself; the program simply carries on and prints a wrong answer. These lines make the failure visible.

THINK

1. Why do we need separate variables `d_a`, `d_b`, `d_c` on the device? Why can the kernel not use `a`, `b`, `c` directly?
2. In `cudaMemcpy(d_a, &a, ...)` we pass `&a`, but in the kernel launch we pass `d_a`. Why?
3. How many threads run the `add` kernel? What would you change to run it with 10 threads, and would the answer change?

Exercise 3 – Vector addition using 512 blocks and one thread

Now we add two vectors of 512 integers. The kernel is launched with **512 blocks of one thread** each. Inside the kernel, `blockIdx.x` tells each thread which element it is responsible for. The vectors are filled with `rand() % 100` on the host.

Colab CODE cell • writes `ex3_blocks.cu`

```
%%writefile ex3_blocks.cu
#include <stdio>
#include <stdlib>

#define N 512

__global__ void add(int *a, int *b, int *c) {
    // each block runs only one addition
    c[blockIdx.x] = a[blockIdx.x] + b[blockIdx.x];
}

void random_ints(int *x, int size) {
    for (int i = 0; i < size; i++)
        x[i] = rand() % 100;
}

int main(void) {
    int *a, *b, *c;           // host copies of a, b, c
    int *d_a, *d_b, *d_c;    // device copies of a, b, c
    int size = N * sizeof(int);

    // Allocate space for device copies of a, b, c
    cudaMalloc((void **)&d_a, size);
    cudaMalloc((void **)&d_b, size);
    cudaMalloc((void **)&d_c, size);

    // Allocate space for host copies of a, b, c and set up input values
    a = (int *)malloc(size); random_ints(a, N);
    b = (int *)malloc(size); random_ints(b, N);
    c = (int *)malloc(size);

    // Copy inputs to device
    cudaMemcpy(d_a, a, size, cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, b, size, cudaMemcpyHostToDevice);

    // Launch add() kernel on GPU with N blocks, 1 thread per block
    add<<<N, 1>>>>(d_a, d_b, d_c);

    cudaError_t err = cudaGetLastError();
    if (err != cudaSuccess)
        printf("Kernel launch failed: %s\n", cudaGetErrorString(err));

    // Copy result back to host
    cudaMemcpy(c, d_c, size, cudaMemcpyDeviceToHost);

    for (int r = 0; r < N; r++)
        printf("%d + %d = %d\n", a[r], b[r], c[r]);

    // Check the GPU's answers against the CPU
    int errors = 0;
    for (int r = 0; r < N; r++)
        if (c[r] != a[r] + b[r]) errors++;
    printf("Verification: %d mismatches out of %d\n", errors, N);
```

Colab CODE cell • writes ex3_blocks.cu

```
// Cleanup
free(a); free(b); free(c);
cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
return 0;
}
```

Colab CODE cell • compiles and runs

```
!nvcc -arch=sm_75 ex3_blocks.cu -o ex3_blocks && ./ex3_blocks
```

The program prints 512 lines. Colab collapses long output, so scroll to the end. The last line must be

Verification: 0 mismatches out of 512. The first lines are:

expected output (first three lines)

```
83 + 77 = 160
86 + 34 = 120
77 + 90 = 167
```

PREDICT BEFORE YOU RUN

How many blocks and how many threads in total does `add<<<N, 1>>>` launch? A GPU executes threads in groups of 32 called *warps*. What does that mean for a block that contains only one thread?

Exercise 4 – Vector addition using one block and 512 threads

The same calculation, but now with **one block of 512 threads**. The only differences from Exercise 3 are the launch configuration `<<<1, N>>>` and the use of `threadIdx.x` to choose the element.

Colab CODE cell • writes ex4_threads.cu

```
%%writefile ex4_threads.cu
#include <stdio>
#include <stdlib>

#define N 512

__global__ void addT(int *a, int *b, int *c) {
    c[threadIdx.x] = a[threadIdx.x] + b[threadIdx.x];
}

void random_ints(int *x, int size) {
    for (int i = 0; i < size; i++)
        x[i] = rand() % 100;
}

int main(void) {
    int *a, *b, *c;           // host copies of a, b, c
    int *d_a, *d_b, *d_c;     // device copies of a, b, c
    int size = N * sizeof(int);

    // Allocate space for device copies of a, b, c
    cudaMalloc((void **)&d_a, size);
    cudaMalloc((void **)&d_b, size);
    cudaMalloc((void **)&d_c, size);

    // Allocate space for host copies of a, b, c and set up input values
    a = (int *)malloc(size); random_ints(a, N);
    b = (int *)malloc(size); random_ints(b, N);
    c = (int *)malloc(size);

    // Copy inputs to device
    cudaMemcpy(d_a, a, size, cudaMemcpyHostToDevice);
```

Colab CODE cell • writes ex4_threads.cu

```

    cudaMemcpy(d_b, b, size, cudaMemcpyHostToDevice);

    // Launch addT() kernel on GPU with 1 block of N threads
    addT<<<1, N>>>(d_a, d_b, d_c);

    cudaError_t err = cudaGetLastError();
    if (err != cudaSuccess)
        printf("Kernel launch failed: %s\n", cudaGetErrorString(err));

    // Copy result back to host
    cudaMemcpy(c, d_c, size, cudaMemcpyDeviceToHost);

    for (int r = 0; r < N; r++)
        printf("%d) %d + %d = %d\n", r, a[r], b[r], c[r]);

    // Check the GPU's answers against the CPU
    int errors = 0;
    for (int r = 0; r < N; r++)
        if (c[r] != a[r] + b[r]) errors++;
    printf("Verification: %d mismatches out of %d\n", errors, N);

    // Cleanup
    free(a); free(b); free(c);
    cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
    return 0;
}

```

Colab CODE cell • compiles and runs

```
!nvcc -arch=sm_75 ex4_threads.cu -o ex4_threads && ./ex4_threads
```

Expected: the first line is 0) 83 + 77 = 160, the last result line is 511) 67 + 6 = 73, and the final line is Verification: 0 mismatches out of 512.

EXPERIMENT – find the limit

In the `ex4_threads.cu` cell change `#define N 512` to `#define N 2048`, run the `%writefile` cell, then compile and run again. What is printed, and which part of the program told you? Which limit of the GPU did you hit? (Hint: the message mentions the launch *configuration*.) Change N back to 512 afterwards.

THINK

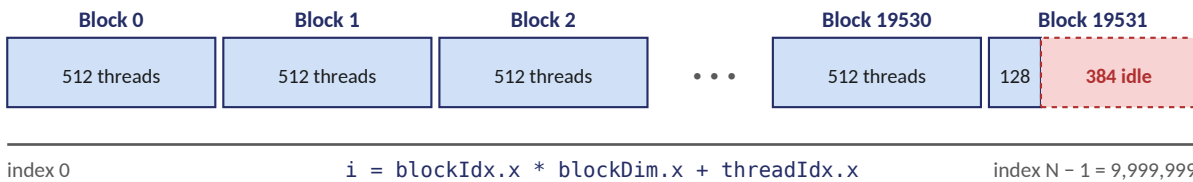
One block of threads cannot cover a very large array. Exercise 5 combines what you learned in Exercises 3 and 4.

Exercise 5 – Multiply two vectors using blocks and threads

Write a CUDA program that multiplies two vectors element by element: $c[i] = a[i] * b[i]$. Fill the arrays `a` and `b` with random values in the same way as Exercise 4.

- ☐ (a) Use an array size of **10,000,000** elements.
- ☐ (b) Use **512 threads per block**, and enough blocks to cover all 10,000,000 elements. You are using blocks and threads together.
- ☐ (c) Print the **last 1000 results** calculated by the kernel.

As you have seen by now, a CUDA kernel normally has **no loop**. Instead of one thread looping over the array, every element gets its own thread, and each thread works out which element is its own.



THE GLOBAL INDEX

With many blocks, `threadIdx.x` only counts 0 to 511 *inside* a block, so it cannot identify an element by itself. Combine all three built-in variables: $i = \text{blockIdx.x} * \text{blockDim.x} + \text{threadIdx.x}$. The number of blocks must be rounded **up**: $(N + \text{threadsPerBlock} - 1) / \text{threadsPerBlock}$.

Complete the starter program below. Each place you must change is marked with **TODO** and highlighted. The starter compiles as it is, but it does not compute anything useful until you finish all the TODOs.

Colab CODE cell · writes `ex5_vecmul.cu` (edit the TODO lines)

```
%%writefile ex5_vecmul.cu
#include <stdio>
#include <stdlib>

#define N 10000000 // 10 million elements
#define THREADS_PER_BLOCK 512

__global__ void vecMul(int *a, int *b, int *c, int n) {
    // TODO 1: global index of this thread (use blockIdx, blockDim, threadIdx)
    int i = 0; // <-- replace this line

    // TODO 2: only compute if i is inside the array. Why is this needed?
    c[i] = a[i] * b[i]; // <-- guard this line
}

void random_ints(int *x, int size) {
    for (int i = 0; i < size; i++)
        x[i] = rand() % 100;
}

int main(void) {
    int *a, *b, *c; // host copies of a, b, c
    int *d_a, *d_b, *d_c; // device copies of a, b, c
    size_t size = (size_t)N * sizeof(int);

    // Allocate device memory
    cudaMalloc((void **)&d_a, size);
    cudaMalloc((void **)&d_b, size);
    cudaMalloc((void **)&d_c, size);

    // Allocate host memory and generate the input values
    a = (int *)malloc(size); random_ints(a, N);
    b = (int *)malloc(size); random_ints(b, N);
    c = (int *)malloc(size);

    // Copy inputs to device
    cudaMemcpy(d_a, a, size, cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, b, size, cudaMemcpyHostToDevice);

    // TODO 3: number of blocks needed to cover N elements (round UP)
    int blocks = 0; // <-- replace this line
    printf("Launching %d blocks x %d threads\n", blocks, THREADS_PER_BLOCK);

    // TODO 4: launch vecMul with <<<blocks, THREADS_PER_BLOCK>>>

    cudaError_t err = cudaGetLastError();
    if (err != cudaSuccess)
        printf("Kernel launch failed: %s\n", cudaGetErrorString(err));
}
```

Colab CODE cell · writes ex5_vecmul.cu (edit the TODO lines)

```
// TODO 5: copy the result d_c back into c on the host

// TODO 6: print the last 1000 results as "index) a x b = c"

// TODO 7: verify all N results on the CPU and print the number of mismatches

// Cleanup
free(a); free(b); free(c);
cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
return 0;
}
```

Colab CODE cell · compiles and runs

```
!nvcc -arch=sm_75 ex5_vecmul.cu -o ex5_vecmul && ./ex5_vecmul
```

Self-check: your program is correct if it reports these numbers

Item	Expected value
Blocks launched (printed by the starter)	19532 blocks × 512 threads
Total threads launched	10,000,384 (that is 384 more than the 10,000,000 elements)
Active threads in the last block	128 (the other 384 must be stopped by your guard)
First of the last 1000 results	9999000) 76 × 52 = 3952
Last result	9999999) 31 × 73 = 2263
Verification	0 mismatches out of 10000000

The numbers depend on generating **a** first and **b** second with the `random_ints` function in the starter. Your print format may differ; compare the numbers.

COMMON MISTAKE

Missing bounds check. 19,532 blocks × 512 threads = 10,000,384 threads, but the array has 10,000,000 elements. Without `if (i < n)` the last 384 threads read and write *past the end* of the arrays. This is undefined behaviour: the program may still print correct-looking results, or it may crash later. Always guard.

Rounding down. $N / 512$ in integer arithmetic gives 19,531 blocks and leaves the last 128 elements unprocessed.

Forgetting the last line of output. Colab collapses long output. Scroll to the bottom before you decide the program is wrong.

THINK

Why is it a good idea to use a multiple of 32 for the threads per block? What happens to the total number of threads launched if you use 500 instead of 512?

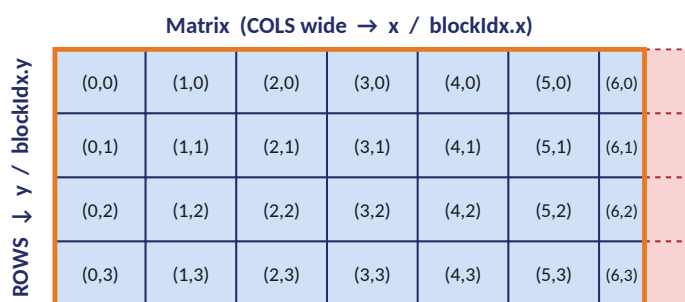
Exercise 6 – Multiply two 10,000 × 10,000 matrices element by element

Extend Exercise 5 to a two-dimensional matrix of **10,000 × 10,000** integers for **A**, **B** and **C**. Multiply the corresponding elements, for example $C[4][3] = A[4][3] \times B[4][3]$. This is **not** matrix multiplication; each output element depends on one element of **A** and one of **B**.

□ Print the last thousand elements, $C[9999][9000]$ to $C[9999][9999]$.

Mapping a 2D grid onto a matrix

CUDA lets you arrange blocks and threads in two dimensions with **dim3**. Threads have an **x** coordinate and a **y** coordinate, and so do blocks. By convention **x runs along the columns** and **y runs down the rows**.



Reading the picture

Blue: threads that map to a real element
 Red, dashed: threads outside the matrix
 (your guard must stop these)

For 10,000 columns (x)

$10,000 / 32 = 312.5 \rightarrow 313$ blocks
 last block column: 16 active, 16 idle

For 10,000 rows (y)

$10,000 / 16 = 625 \rightarrow 625$ blocks

Each tile is one block of $32 \times 16 = 512$ threads, labelled (blockIdx.x, blockIdx.y).

Three pieces of code work together. In the kernel, use the same idea as Exercise 5 once for each dimension, then convert the pair (row, col) into one position in a flat array. On the host, a **dim3** variable gives the number of blocks in each dimension:

What	Code
Column of this thread	<code>col = blockIdx.x * blockDim.x + threadIdx.x</code>
Row of this thread	<code>row = blockIdx.y * blockDim.y + threadIdx.y</code>
Flat (row-major) index	<code>idx = row * cols + col</code>
Threads per block	<code>dim3 threadsPerBlock(32, 16);</code> → $32 \times 16 = 512$ threads
Blocks in the grid	<code>dim3 numBlocks(blocks_in_x, blocks_in_y);</code> (round both up)
Launch	<code>mulElem<<<numBlocks, threadsPerBlock>>>(...)</code>

COMMON MISTAKE

Do not declare the matrix as a local array such as `int A[10000][10000];`. It needs 400 MB, which overflows the stack and crashes the program. Use `malloc` for one flat array of `ROWS * COLS` integers, as the starter does.

Mixing up x and y. The x coordinate selects the *column* and the y coordinate selects the *row*. If you swap them the program may still run, but the elements you print will be wrong.

Guarding only one dimension. Both `row < rows` and `col < cols` must be checked.

Memory. Each matrix is 400 MB, so the three matrices take 1.2 GB on the GPU and another 1.2 GB on the host. Do not create extra copies. If Colab says the session crashed after using all available RAM, restart the runtime and check the size of your arrays.

Complete the starter program. Each place you must change is marked with **TODO**.

Colab CODE cell • writes ex6_matrix.cu (edit the TODO lines)

```

%%writefile ex6_matrix.cu
#include <stdio>
#include <stdlib>

#define ROWS 10000
#define COLS 10000

// Element-wise product: C[r][c] = A[r][c] * B[r][c] (NOT matrix multiplication)
__global__ void mulElem(int *a, int *b, int *c, int rows, int cols) {
    // TODO 1: row comes from the y dimension, col from the x dimension
    int row = 0; // <-- replace this line
    int col = 0; // <-- replace this line

    // TODO 2: only compute if BOTH row and col are inside the matrix
    // TODO 3: convert (row, col) into a flat index: row * cols + col
    int idx = 0; // <-- replace this line
    c[idx] = a[idx] * b[idx]; // <-- guard this line
}

void random_ints(int *x, size_t size) {
    for (size_t i = 0; i < size; i++)
        x[i] = rand() % 100;
}

int main(void) {
    int *a, *b, *c; // host copies (flat arrays, row-major)
    int *d_a, *d_b, *d_c; // device copies
    size_t count = (size_t)ROWS * COLS;
    size_t size = count * sizeof(int); // 400 MB per matrix

    cudaMalloc((void **)&d_a, size);
    cudaMalloc((void **)&d_b, size);
    cudaMalloc((void **)&d_c, size);

    a = (int *)malloc(size); random_ints(a, count);
    b = (int *)malloc(size); random_ints(b, count);
    c = (int *)malloc(size);

    cudaMemcpy(d_a, a, size, cudaMemcpyHostToDevice);
    cudaMemcpy(d_b, b, size, cudaMemcpyHostToDevice);

    // 32 x 16 = 512 threads per block
    dim3 threadsPerBlock(32, 16);
    // TODO 4: blocks needed in x (columns) and in y (rows), rounding UP
    dim3 numBlocks(1, 1); // <-- replace this line
    printf("Grid: %d x %d blocks\n", numBlocks.x, numBlocks.y);

    // TODO 5: launch mulElem with <<<numBlocks, threadsPerBlock>>>

    cudaError_t err = cudaGetLastError();
    if (err != cudaSuccess)
        printf("Kernel launch failed: %s\n", cudaGetErrorString(err));

    // TODO 6: copy d_c back into c on the host

    // TODO 7: print C[9999][9000] ... C[9999][9999] (the last 1000 elements)

    // TODO 8: verify all elements on the CPU and print the number of mismatches

    free(a); free(b); free(c);
    cudaFree(d_a); cudaFree(d_b); cudaFree(d_c);
    return 0;
}

```

Colab CODE cell · compiles and runs

```
!nvcc -arch=sm_75 ex6_matrix.cu -o ex6_matrix && ./ex6_matrix
```

Self-check: your program is correct if it reports these numbers

Item	Expected value
Grid (printed by the starter)	Grid: 313 x 625 blocks (195,625 blocks)
Total threads launched	100,160,000 (that is 160,000 more than the 100,000,000 elements)
C[9999][9000]	29 x 56 = 1624
C[9999][9999]	86 x 21 = 1806
Verification	0 mismatches out of 100000000

 As in Exercise 5, these values assume **a** is generated first and **b** second with the starter's `random_ints`.

FINISHING OFF

- ☐ Your notebook has a text heading and the working cells for every exercise, with the outputs visible.
- ☐ Use **File** → **Save** (Colab also saves to Drive automatically).
- ☐ Submit or share the notebook as your lecturer instructs (**File** → **Download** → **Download .ipynb** or **Share**).
- ☐ Choose **Runtime** → **Disconnect and delete runtime** to release the GPU.

Quick reference

Item	What it does
<code>__global__</code>	Marks a function as a kernel : it runs on the GPU and is launched from the CPU. It must return void .
<code>cudaMalloc((void **)&d_p, bytes)</code>	Allocates <i>bytes</i> of GPU memory and stores the address in d_p .
<code>cudaMemcpy(dst, src, bytes, kind)</code>	Copies memory. kind is <code>cudaMemcpyHostToDevice</code> or <code>cudaMemcpyDeviceToHost</code> .
<code>cudaFree(d_p)</code>	Releases GPU memory allocated with <code>cudaMalloc</code> .
<code>kernel<<<blocks, threads>>>(args)</code>	Launches the kernel with the given number of blocks and threads per block. Both can be a dim3 for 2D/3D.
<code>cudaGetLastError()</code>	Returns the error from the most recent launch or CUDA call, or cudaSuccess .
<code>cudaGetErrorString(err)</code>	Turns an error code into readable text.
<code>cudaDeviceSynchronize()</code>	Makes the CPU wait until all GPU work has finished (needed before the program ends if the kernel uses <code>printf</code>).
<code>threadIdx.x / .y</code>	Position of the thread <i>inside</i> its block.
<code>blockIdx.x / .y</code>	Position of the block inside the grid.
<code>blockDim.x / .y</code>	Number of threads per block in that dimension.
<code>gridDim.x / .y</code>	Number of blocks in the grid in that dimension.

Troubleshooting in Colab

What you see	Likely cause and fix
<code>nvidia-smi</code> fails or says it cannot communicate with the driver	The runtime has no GPU. Runtime → Change runtime type , choose T4 GPU, save, and run the cell again.
<code>nvcc: command not found</code>	Use the full path <code>/usr/local/cuda/bin/nvcc</code> in the compile command.
Nothing happens, or an error about <code>%%writefile</code>	<code>%%writefile</code> is not on the first line of the cell. Remove any blank line or comment above it.
<code>no kernel image is available for execution on the device</code> or <code>Kernel launch failed</code>	The <code>-arch</code> flag does not match your GPU. Recheck Step 5 and recompile.
<code>Kernel launch failed: invalid configuration argument</code>	Too many threads per block (the limit is 1024), or a zero-sized grid. In Exercise 5 a value of 0 blocks (the starter's default) causes this.
Your edit has no effect	You changed the code but did not re-run the <code>%%writefile</code> cell and then the compile cell.
Results look right but the last few are wrong or zero	The block count was rounded down, or the guard uses the wrong limit.
<code>error: identifier ... is undefined</code> or other compile errors	Read the first error only; it usually names the line. Check spelling (<code>blockIdx</code> , <code>threadIdx</code>) and missing semicolons.
"Your session crashed after using all available RAM" (Exercise 6)	The matrices were declared as local arrays or copied unnecessarily. Use flat <code>malloc</code> arrays; restart the runtime.
Runtime disconnected and files are gone	Colab's disk is temporary. Reconnect and choose Runtime → Run all .
"Cannot connect to GPU backend"	The free GPU quota may be used up for now. Wait and try later; do not buy compute units for this lab.